What CKS Inherits from Knowledge Objects (KO) Alone: The Cost Model and the Separation-of-Concerns Architecture as Standalone Architectural Inheritance

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 5, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to specialize, in a single derivation, what CKS inherits from Knowledge Objects (KO; Zahn & Chana 2026) considered alone — that is, treating the KO-inheritance commitment as a standalone architectural posture independent of CKS's adjacent inheritance from OIDA. The note pairs with a companion treatment of the joint KO/OIDA inheritance relationship and with a sibling treatment of OIDA inheritance considered alone; this note covers the KO portion.

Abstract

The CKS pattern's positioning against Knowledge Objects is developed at §6.2 of the source paper and consolidated in the foundational companion note *Inheritance and Extension: How CKS Builds on Knowledge Objects and OIDA Along the Multi-Human Axis* (Li, 27 April 2026). KO is the closest empirical precedent for the CKS substrate at the cost-model layer, and the source paper makes the inheritance precise: CKS inherits from KO two commitments — the database-like cost model that KO validated empirically at ~252× per-query cost difference at N=7,000, and the separation-of-concerns architecture between external addressable storage and LLM processing. CKS extends KO on two axes — governance semantics, and where disambiguation work lives — with the extension axes leaving the inherited commitments intact. This note formalizes the KO-inheritance posture as standalone: states the two inherited commitments precisely, names what KO inheritance does not include, distinguishes inheritance at the architectural-pattern level from inheritance at the implementation-detail level, develops the two-axis extension as the structure within which the inheritance is preserved, identifies the failure modes that mis-attribute KO inheritance, and provides an operational test for whether a given system's KO-inheritance commitment is CKS-coherent specifically.

1. Why the KO-inheritance posture needs to be formalized as standalone

The companion foundational note *Inheritance and Extension* treats the joint KO/OIDA inheritance together and identifies the multi-human axis as the scope along which CKS's already-distinct posture extends. This note specializes the KO portion. KO inheritance has independent architectural content — the cost model and the separation-of-concerns architecture — that traces specifically to KO rather than to OIDA or to other prior-art sources, and the architectural content is load-bearing for several CKS commitments downstream.

The cost-curve regime KO supplies is load-bearing for the linear-cost commitment (§6.1, §6.3), for the source-of-truth commitment (§11.3), and for the AI-as-substrate-mediator commitment (§4.2). Each rests on the KO-validated cost regime as the empirical backbone the source paper inherits. Without precise specification of what is inherited from KO, the cost regime appears either as a CKS innovation — mis-crediting the empirical work that established it — or as an undifferentiated background assumption, which obscures the architectural lineage on which the downstream commitments rest. The standalone treatment is what makes the KO-specific inheritance sharp.

A second motivation is the architectural-difference posture §9.4 protects: that CKS is *architecturally different from OIDA on the multi-human axis, not OIDA plus multi-human*. That posture depends on precise specification of what is inherited from each prior-art source and what is fresh in CKS. The KO-inheritance lineage publicly formalized as standalone is what lets the architectural-difference claim rest on a clean inheritance baseline rather than reconstruct it.

2. The two inherited commitments

The source paper at §6.2 names the inheritance and the extension in precise terms. Two commitments carry forward.

Commitment 1 — The database-like cost model. O(1) retrieval via hash lookup, per-query token cost approximately constant from N=100 through N=100,000, and a ~252× per-query cost difference at N=7,000 versus in-context memory that, in KO's language, "grow[s] linearly as the corpus expands." The cost-curve shape is consistent across the adjacent voices the source paper cites: Karpathy's LLM Wiki pattern, with reported efficiency advantages for some use cases; Oracle's 2026 developer article on files versus databases for agent memory, concluding that databases are preferable for structured, high-cardinality, query-heavy memories due to indexed access, concurrency, and predictable scaling; and 2026 industry guidance on knowledge base / knowledge graph backends for LLM systems, recommending KB/KG as authoritative source of truth with the LLM as natural-language frontend and structured lookups for fact retrieval. The cost-curve shape across this evidence — linear in storage, O(1) or O(log N) in lookup, near-constant in per-query tokens — is the regime CKS commits to by design, inheriting the empirical backbone KO supplies.

Commitment 2 — The separation-of-concerns architecture between external storage and LLM processing. Facts live in addressable external storage where they cannot decay through compression or be lost at session boundaries; the LLM retains query understanding, answer generation, and the extraction of new content from ongoing interaction. KO names the failure mode this separation prevents — *context rot*, with the named subfailures of capacity overflow, compaction loss, and goal

drift under repeated compression passes. CKS adopts the *context rot* vocabulary as the named failure mode the substrate-as-source-of-truth commitment defends against (§11.3, and the companion derivation *The Substrate Is the Source of Truth*).

The two commitments together specify the architectural posture KO contributes. A KO-coherent system commits to both: facts live in external addressable storage, retrieval is database-like, and the LLM processes queries against that storage rather than maintaining the storage internally. CKS inherits both wholesale at the architectural-pattern level.

3. What KO inheritance does NOT include

The two commitments named in §2 are scoped narrowly. Several KO features that surround the inherited commitments do not carry forward, and naming each is what keeps the KO-inheritance posture from being overstated.

Not KO's specific data models or type systems. KO commits to typed Knowledge Objects, hash addressing, density-adaptive retrieval with learned thresholds (τ=0.85 in KO's experiments), and `(subject, predicate)` structured-key matching as fallback. These are KO design choices CKS treats as implementation territory it does not commit to. CKS substrate schemas are deployment choices, not inheritance.

Not KO's `provenance_metadata` field. KO commits to a single `provenance_metadata` field as its governance hook — adequate for attribution and audit at the retrieval-unit level. CKS extends this on the governance-semantics axis (per §5 below), but the single-field commitment itself is not what carries forward.

Not KO's retrieval-algorithm choices. Density-adaptive retrieval, learned thresholds, and the structured-key fallback are KO's choices about where disambiguation work lives — they sit on the extension axis where CKS specifically diverges from KO. They are not part of the inherited commitment set.

Not KO's silence on substrate authority. KO is silent on whether the substrate is governed by humans in the authority sense; the architectural framing is about storage and processing, not about authority over substrate content. The CKS human-governed commitment is not KO inheritance; it is fresh in CKS, and the governance-semantics extension axis is what makes the difference visible.

Not KO's silence on cell decomposition. KO frames the LLM as substrate client consuming structured memory to answer queries. KO does not commit to a substrate-and-cell decomposition pattern, to cells as operational units, or to orchestration rules governing cell behavior over substrate. The substrate-cell boundary commitment, defended at §2.1 and §4.1 of the source paper and in the companion note *State and Execution: A Precise Definition of the Substrate-Cell Boundary*, is CKS's own architectural move and does not trace to KO.

The KO-inheritance posture is the cost regime and the separation of concerns. The other architectural moves CKS makes around the substrate are CKS's own commitments, not KO inheritance.

4. Architectural-pattern level, not implementation level

The KO inheritance is at the architectural-pattern level, not at the implementation level. A CKS-coherent system inherits the cost regime and the separation-of-concerns architecture as architectural commitments, regardless of whether the system shares any code, library, or runtime with KO implementations. A KO-inspired system written from scratch satisfies the inheritance commitment if it instantiates the two commitments named in §2; conversely, a system that imports KO code but operates under a cost regime or processing architecture that violates either commitment fails the inheritance commitment despite the implementation lineage.

The level distinction matters because it shapes what counts as evidence for inheritance. Architectural-pattern-level inheritance is verified by checking whether the system's cost regime and processing architecture satisfy the two commitments — the empirical cost curve is approximately constant per query as N grows, and facts live in external addressable storage rather than in the LLM's per-session context. Implementation-level lineage, by contrast, is neither necessary nor sufficient.

The selectivity is also at the implementation-detail level. KO-specific implementation choices — the `provenance_metadata` field, the density-adaptive retrieval mechanism, the τ=0.85 threshold, the structured-key matching algorithm — do not carry forward; the architectural-pattern-level commitments do. A CKS deployment is free to choose its own substrate schema, retrieval mechanism, and disambiguation strategy without violating the inheritance, provided the cost regime and the separation-of-concerns architecture are preserved.

5. The two-axis extension within which inheritance is preserved

Per §6.2, CKS extends KO on two axes. The extension does not remove or modify the inherited commitments; it adds architectural content to the substrate while the cost regime and the separation-of-concerns architecture remain intact.

First axis — governance semantics. Where KO commits to a single `provenance_metadata` field as its governance hook, CKS encodes role and authority, approval states, decision rationale, and conflict state as first-class substrate content. The CKS substrate carries governance-relevant structure as schema, not as a single metadata field. This extension is additive at the substrate level — the substrate carries more structure — and does not change the cost regime or the separation-of-concerns architecture. The companion derivations *Authority, Not Labor* and *Path Retraceability and the Accountability Vocabulary* develop the substrate-level content this axis names.

Second axis — where disambiguation work lives. KO disambiguates computationally at retrieval time, via density-adaptive retrieval and structured-key matching; CKS disambiguates through human-authored schema at the cell level, making the distinctions humans draw into substrate content encoded at authoring time rather than recovered by runtime algorithm. The two design stances reflect different philosophies about what the substrate is responsible for carrying. KO treats the substrate as storage over which algorithmic disambiguation operates; CKS treats the substrate as the authored artifact whose structure already carries the distinctions. This extension is additive at the schema level — the substrate carries the distinctions humans draw — and does not change the cost regime or the separation-of-concerns architecture.

The two axes together are what §6.2 means by *CKS extends KO on two axes*. The phrase preempts the reading "CKS is KO with extra fields": the additional schema content exists because CKS commits to a different stance about where disambiguation and governance work sit — explicit in authored structure rather than implicit in retrieval algorithm or in a single attribution field. The extra fields are not decoration; they are substrate content doing disambiguation work CKS's design philosophy requires to be explicit rather than computed.

6. Failure modes that mis-attribute KO inheritance

Each failure mode names a way an implementation can mis-position the inheritance without realizing it.

Over-attribution. The implementation presents CKS commitments as KO inheritance when they are fresh in CKS — for example, treating the human-governed commitment, the substrate-and-cell decomposition, the conflict-as-first-class commitment, or the architectural-governance qualifier as KO inheritance. Over-attribution weakens CKS's prior-art posture by attributing fresh contributions to inheritance, and it mis-positions the architectural-difference claim because the difference is understated.

Under-attribution. The implementation presents the cost regime or the separation-of-concerns architecture as fresh in CKS when both trace to KO. Under-attribution weakens credibility — reviewers identifying the inheritance from the empirical cost-curve evidence — and makes the downstream commitments harder to defend, because their empirical backbone is presented as fresh rather than as inherited.

Implementation-level conflation. The implementation presents inheritance as requiring specific KO codebases, libraries, or runtimes. This is not the architectural-pattern-level inheritance the architecture commits to. A KO-inspired CKS system written from scratch satisfies the inheritance, and a system that imports KO code but operates under a different cost regime does not.

Selective-inheritance distortion. The implementation inherits one of the two commitments while abandoning the other — for example, adopts external addressable storage but operates under a cost regime that grows superlinearly per query as N grows, because retrieval re-reads the full corpus per query; or adopts the cost regime but materializes facts inside the LLM's context window rather than in external addressable storage. The architectural commitment is to both inherited commitments together; selective inheritance produces architectures that claim KO lineage but do not satisfy the inheritance.

Adjacent-pattern conflation. The implementation presents inheritance from RAG, parametric memory, or in-context memory as KO inheritance. KO is distinct from each: RAG retrieves over an unstructured corpus rather than over typed external addressable storage; parametric memory lives inside the LLM's weights rather than outside the model; in-context memory lives in the per-session window rather than in persistent external storage. The companion derivation on the three adjacencies treats these distinctions in detail; under the KO-inheritance commitment, the cost regime and the separation-of-concerns architecture are what specifically traces to KO, not features shared with adjacent patterns.

Extension-axis distortion. The implementation extends KO on axes the source paper does not name as the CKS extension axes — for example, on the storage-format axis (graph vs. relational vs. document) or on the retrieval-algorithm axis (semantic vs. keyword vs. hybrid). These are deployment choices CKS does not commit to either way; they are not the extension axes the architecture names. The two CKS extension axes are governance semantics and where disambiguation work lives.

7. Operational test

A system instantiates the KO-inheritance commitment if and only if all of the following are true.

1. **Cost regime.** The system's cost regime is database-like: O(1) or O(log N) retrieval, per-query token cost approximately constant as N grows, and per-query cost differences from in-context alternatives that grow with N rather than shrinking.
2. **Storage / processing separation.** Facts live in addressable external storage where they cannot decay through compression or be lost at session boundaries; the LLM retains query understanding, answer generation, and new-content extraction but does not maintain the storage internally.
3. **Architectural-pattern level.** The system instantiates (1) and (2) regardless of codebase or library lineage; the inheritance commitment is verified at the architectural level, not at the code level.
4. **Selective at implementation detail.** KO-specific implementation choices — the `provenance_metadata` field, the density-adaptive retrieval mechanism, the τ=0.85 threshold, the `(subject, predicate)` structured-key matching algorithm — are not required, while the two architectural-pattern-level commitments are.
5. **Two-axis extension preserved.** Governance semantics is encoded as first-class substrate content rather than as a single metadata field, and disambiguation work is located in human-authored schema rather than in a runtime algorithm.

A system that fails any of (1)–(5) does not instantiate the KO-inheritance commitment in the architectural sense the source paper names. A system that satisfies (1)–(2) but not (3)–(5) is in the broad inheritance lineage of KO but does not instantiate the CKS architectural posture at the inheritance-specialization scope.

8. Why naming KO inheritance as standalone matters

Implementations under pressure to position CKS against prior art consistently drift toward either over-attributing inheritance — presenting CKS as KO with extra fields — or under-attributing inheritance — obscuring the empirical and architectural lineage on which the downstream cost commitments rest. Both drifts produce inaccurate positioning that obscures what CKS contributes architecturally and what it inherits.

Implementations that drift toward over-attributing produce systems that claim too little architectural difference from KO; the architectural-difference posture the source paper protects becomes harder to defend, because the difference is understated and the extra schema content reads as decoration rather than as the substrate-level commitment §6.2 names. Implementations that drift toward under-attributing produce systems that claim too much architectural novelty; the prior-art credibility suffers when

reviewers identify the inheritance from the cost-curve evidence the source paper cites, and the downstream commitments — linear-cost scaling, source-of-truth, AI-as-substrate-mediator — appear as fresh CKS contributions when they rest on the KO-validated regime.

Naming KO inheritance as a standalone architectural commitment — with the two inherited commitments specified in §2, the limitations clarified in §3, the architectural-pattern level distinguished from the implementation level in §4, the two-axis extension developed in §5, and the failure modes named in §6 — gives downstream readers a precise specification of what CKS inherits from KO architecturally. Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should address the inheritance relationship as formalized here. Subsequent work that uses the inheritance relationship differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

Companion notes

Li, W. (2026). *Inheritance and Extension: How CKS Builds on Knowledge Objects and OIDA Along the Multi-Human Axis.* 27 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Authority, Not Labor: A Precise Definition of "Human-Governed" in the Coordination Knowledge Substrate Pattern.* 24 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *State and Execution: A Precise Definition of the Substrate-Cell Boundary in the Coordination Knowledge Substrate Pattern.* 25 April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *What CKS Inherits from Knowledge Objects (KO) Alone: The Cost Model and the Separation-of-Concerns Architecture as Standalone Architectural Inheritance.* May 5, 2026. ORCID: 0009-0004-8065-3235.